

Подробный конспект с примерами: ACID, изоляция транзакций, блокировки, MVCC и индексы

Теория, лабораторные сценарии, SQL и вопросы для собеседования

Подготовлено: август 2026

Рекомендации

по решению проблем

Что добавлено

Справа от основной страницы размещены рекомендации по устранению описанных проблем. Формулировки короткие, чтобы ими можно было пользоваться как чек-листом при разработке и на собеседовании.

Содержание

Часть 0. Учебный стенд

0.1. Базовая схема

0.2. Набор данных для экспериментов с индексами

0.3. Две параллельные сессии

Часть 1. Свойства ACID (кратко)

Часть 2. Аномалии конкурентного доступа (кратко)

2.1. Dirty read: что было бы в СУБД, которая его допускает

2.2. Non-repeatable read: изменилась та же строка

2.3. Phantom read: изменился состав множества

2.4. Lost update: опасная схема “прочитал - посчитал - записал”

2.5. Write skew: разные строки, общий инвариант

Часть 3. Уровни изоляции — углублённо

3.1. Идея и синтаксис

3.2. Read Uncommitted

3.3. Read Committed (по умолчанию)

3.4. Repeatable Read

3.5. Serializable — сильнейший

3.6. Сводная таблица

Часть 4. Блокировки, MVCC и дедлоки — углублённо

4.1. Двухфазная блокировка (2PL)

4.2. Типы и режимы блокировок в PostgreSQL

4.3. MVCC изнутри

4.4. Дедлоки

4.5. Пессимистичный vs оптимистичный подход

Часть 5. Индексы — углублённо

5.1. Зачем и что хранит индекс

5.2. B-дерево изнутри

5.3. Способы сканирования по индексу

5.4. Составные и частичные индексы

5.5. Типы индексов PostgreSQL (что и когда)

5.6. Обслуживание индексов

5.7. Ключевые интуиции (чтобы не зубрить)

Часть 6. Практические задания с решениями

Часть 7. Вопросы на собеседовании и короткие ответы

Часть 8. Итоговая шпаргалка

Источники и актуальные уточнения

Главы 3–5 (уровни изоляции, блокировки/MVCC, индексы) раскрыты углублённо — с внутренним устройством и практикой PostgreSQL. Упор на *почему оно так работает*.

Основа документа сохранена из исходного подробного конспекта. Новые блоки помечены словами **Практический пример**, **Эксперимент**, **Разбор** или **Задание**. Примеры рассчитаны на PostgreSQL 18 и предназначены для запуска в двух терминалах `psql`.

Рекомендации

по решению проблем

Где искать решения

Аномалии и выбор изоляции: части 2-3.
Блокировки, MVCC и дедлоки: часть 4.
Проблемы индексов и планы: часть 5.
Готовые сценарии: часть 6.

Как работать с конспектом

Сначала прочитайте теорию раздела и сформулируйте механизм своими словами.
Затем воспроизведите пример в PostgreSQL. Для конкурентных сценариев откройте две сессии `psql`.
Перед индексными экспериментами выполняйте `ANALYZE`, а планы смотрите через `EXPLAIN (ANALYZE, BUFFERS)`.
Не запоминайте конкретные значения `cost` и времени: они зависят от данных, кэша и машины. Запоминайте форму плана и причину выбора узла.

Рекомендации

по решению проблем

Как закреплять материал

После каждого проблемного сценария воспроизведите его в двух сессиях, затем измените один параметр: уровень изоляции, порядок блокировок или форму SQL. Объясните, почему результат изменился.

Часть 0. Учебный стенд

0.1. Базовая схема

```
DROP SCHEMA IF EXISTS db_notes CASCADE;
CREATE SCHEMA db_notes;
SET search_path TO db_notes, public;

CREATE TABLE accounts (
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  owner       text NOT NULL UNIQUE,
  balance     numeric(12, 2) NOT NULL CHECK (balance >= 0),
  version     integer NOT NULL DEFAULT 0
);

INSERT INTO accounts(owner, balance)
VALUES ('Alice', 1000), ('Bob', 1000), ('Carol', 500);

CREATE TABLE doctors (
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name        text NOT NULL,
  on_call     boolean NOT NULL DEFAULT true
);

INSERT INTO doctors(name, on_call)
VALUES ('Иван', true), ('Мария', true);

CREATE TABLE orders (
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id bigint NOT NULL,
  status      text NOT NULL CHECK (status IN ('new', 'paid', 'cancelled')),
  email       text NOT NULL,
  total       numeric(12, 2) NOT NULL,
  created_at  timestampz NOT NULL DEFAULT now(),
  payload     jsonb NOT NULL DEFAULT '{}'::jsonb
);
```

0.2. Набор данных для экспериментов с индексами

Небольшая таблица часто помещается в несколько страниц, поэтому планировщик может справедливо выбрать Seq Scan, даже если индекс существует. Для наглядных планов создадим 200 000 заказов.

```
INSERT INTO orders(customer_id, status, email, total, created_at, payload)
SELECT
  1 + (random() * 9999)::int,
  (ARRAY['new', 'paid', 'cancelled'])[1 + (random() * 2)::int],
  'user' || g || '@example.com',
  round((10 + random() * 5000)::numeric, 2),
  now() - random() * interval '365 days',
  jsonb_build_object(
    'channel', (ARRAY['mobile', 'web', 'partner'])[1 + (random() * 2)::int],
    'country', (ARRAY['RU', 'KZ', 'BY', 'AM'])[1 + (random() * 3)::int]
  )
FROM generate_series(1, 200000) AS g;

ANALYZE orders;
```

0.3. Две параллельные сессии

Откройте два терминала:

```
psql -d test_db
```

В обоих выполните:

```
SET search_path TO db_notes, public;
```

Далее команды с пометкой **Сессия А** и **Сессия В** выполняйте строго по шагам. Если команда “зависла”, это часто и есть ожидаемое ожидание блокировки.

Рекомендации

по решению проблем

Безопасность стенда

Запускайте DROP SCHEMA и массовую генерацию данных только в учебной БД. Перед экспериментами фиксируйте исходное состояние, после них делайте ROLLBACK или повторно создавайте схему.

Планы запросов

После загрузки данных выполняйте ANALYZE. Сравнивайте не отдельные миллисекунды, а тип узла, actual rows, buffers и число лишних строк.

Часть 1. Свойства ACID (кратко)

A — Atomicity (атомарность). Транзакция — «всё или ничего». Любой сбой или ROLLBACK откатывает все изменения. Механика: WAL (write-ahead log) — сначала намерение в журнал, потом данные; при сбое по журналу откатываем незавершённое.

C — Consistency (согласованность). Транзакция переводит БД из корректного состояния в корректное; все инварианты (PK, FK, CHECK, NOT NULL, UNIQUE, триггеры) соблюдены. Нарушение → откат. Это не consistency из CAP-теоремы.

I — Isolation (изоляция). Параллельные транзакции не «протекают» друг в друга; идеал — эффект последовательного выполнения. Самое дорогое и чаще всего ослабляемое свойство → отсюда уровни изоляции (Часть 3).

D — Durability (устойчивость). После COMMIT результат переживёт любой сбой. Механика: перед подтверждением commit'a записи WAL физически сброшены на диск (fsync).

Практический пример: денежный перевод

Задача: перевести 100 рублей от Alice к Bob. Обе операции должны быть одной транзакцией.

```
BEGIN;

UPDATE accounts
SET balance = balance - 100
WHERE owner = 'Alice';

UPDATE accounts
SET balance = balance + 100
WHERE owner = 'Bob';

COMMIT;
```

Что здесь показывает ACID:

Atomicity: если второе обновление завершится ошибкой, можно сделать ROLLBACK, и списание также исчезнет.

Consistency: CHECK (balance >= 0) не позволит получить отрицательный баланс.

Isolation: другая транзакция не должна увидеть промежуточное состояние, где деньги уже списаны, но ещё не зачислены.

Durability: после успешного COMMIT изменения должны пережить перезапуск сервера.

Ошибка внутри транзакции

```
BEGIN;
UPDATE accounts SET balance = balance - 5000 WHERE owner = 'Alice';
-- ERROR: нарушено ограничение CHECK balance >= 0
ROLLBACK;
```

После ошибки PostgreSQL переводит транзакцию в состояние aborted: до ROLLBACK обычные запросы выполнять нельзя.

SAVEPOINT: частичный откат внутри транзакции

```
BEGIN;
UPDATE accounts SET balance = balance - 50 WHERE owner = 'Alice';

SAVEPOINT before_bonus;
UPDATE accounts SET balance = balance + 100 WHERE owner = 'Unknown';
-- 0 строк обновлено: бизнес-логика может решить отменить только этот шаг
ROLLBACK TO SAVEPOINT before_bonus;

UPDATE accounts SET balance = balance + 50 WHERE owner = 'Bob';
COMMIT;
```

SAVEPOINT не делает вложенную независимую транзакцию: окончательная устойчивость всё равно наступает только после внешнего COMMIT.

Рекомендации

по решению проблем

Ошибка внутри транзакции

После SQL-ошибки сразу выполняйте ROLLBACK или ROLLBACK TO SAVEPOINT. Не продолжайте обычные запросы в aborted-транзакции.

Денежный перевод

Проверяйте число изменённых строк и держите списание с зачислением в одной короткой транзакции. При любой ошибке откатывайте весь перевод.

SAVEPOINT

Используйте только для действительно необязательных подшагов. SAVEPOINT не заменяет внешнюю транзакцию и не делает побочный эффект устойчивым до COMMIT.

Часть 2. Аномалии конкурентного доступа (кратко)

Dirty read — чтение *незафиксированного* чужого изменения (T2 может откатиться).

Non-repeatable read — *та же строка* прочитана дважды и изменилась между чтениями (T2 закоммитила UPDATE).

Phantom read — изменился *состав множества*: по одному условию появились/исчезли строки (T2 закоммитила INSERT/DELETE).

Lost update (сверх ANSI) — два $x = x + 1$ дают +1 вместо +2, одно обновление затёрто.

Write skew (сверх ANSI) — обе транзакции по отдельности корректны, но вместе нарушают инвариант.

Различие в одной фразе: грязное = *незакоммиченное*; неповторяющееся = *та же строка изменилась*; фантом = *состав набора изменился*.

2.1. Dirty read: что было бы в СУБД, которая его допускает

Логический сценарий:

T1 меняет баланс с 1000 на 0, но не фиксирует транзакцию.

T2 читает 0.

T1 делает ROLLBACK.

T2 приняла решение на основе значения, которого никогда не существовало в зафиксированной истории.

В PostgreSQL такой эксперимент не воспроизводится: даже `READ UNCOMMITTED` работает как `READ COMMITTED`, поэтому T2 увидит последнюю зафиксированную версию.

2.2. Non-repeatable read: изменилась та же строка

Сессия A

```
BEGIN ISOLATION LEVEL READ COMMITTED;
SELECT balance FROM accounts WHERE owner = 'Alice'; -- допустим, 1000
```

Сессия B

```
BEGIN;
UPDATE accounts SET balance = 900 WHERE owner = 'Alice';
COMMIT;
```

Сессия A

```
SELECT balance FROM accounts WHERE owner = 'Alice'; -- уже 900
COMMIT;
```

Одна и та же строка в одной транзакции прочитана дважды с разным результатом.

2.3. Phantom read: изменился состав множества

Сессия A

```
BEGIN ISOLATION LEVEL READ COMMITTED;
SELECT count(*) FROM orders WHERE status = 'new';
```

Сессия B

```
INSERT INTO orders(customer_id, status, email, total)
VALUES (42, 'new', 'phantom@example.com', 100);
```

Сессия A

```
SELECT count(*) FROM orders WHERE status = 'new'; -- значение увеличилось
COMMIT;
```

Изменилась не конкретная ранее прочитанная строка, а множество строк, удовлетворяющих условию.

2.4. Lost update: опасная схема “прочитал - посчитал - записал”

Плохой прикладной алгоритм:

```
T1: прочитал balance = 1000
T2: прочитал balance = 1000
T1: записал 900
T2: записал 800
```

Итог 800, хотя ожидалось 700. Обновление T1 потеряно.

Рекомендации

по решению проблем

Non-repeatable / phantom

Когда внутри одной транзакции нужна стабильная картина, используйте REPEATABLE READ. Для сложного общего инварианта переходите на SERIALIZABLE или явные блокировки.

Lost update

Не используйте схему 'прочитал - посчитал - записал'. Делайте атомарный UPDATE ... SET $x = x + \text{delta}$ с условием и RETURNING либо применяйте optimistic/pessimistic locking.

Dirty read

В PostgreSQL отдельное лечение не требуется: MVCC не показывает чужие незакоммиченные версии. Всё равно не полагайтесь на READ UNCOMMITTED как на особый режим.

Безопаснее выполнять вычисление непосредственно в SQL:

```
UPDATE accounts
SET balance = balance - 100
WHERE owner = 'Alice';
```

PostgreSQL сериализует конкурирующие изменения одной строки через row lock, а выражение вычисляется для актуальной версии строки.

2.5. Write skew: разные строки, общий инвариант

Инвариант: хотя бы один врач должен оставаться дежурным.

Обе транзакции видят двух дежурных и выключают разные строки:

```
T1 читает: Иван=true, Мария=true -> выключает Ивана
T2 читает: Иван=true, Мария=true -> выключает Марию
```

Строки разные, прямого write-write конфликта нет. Итог: оба врача false, общий инвариант нарушен. Именно поэтому отсутствие lost update ещё не означает сериализуемость.

Рекомендации

по решению проблем

Write skew

Блокировка одной изменяемой строки не защищает общий инвариант. Используйте SERIALIZABLE либо заранее блокируйте все строки, участвующие в проверке.

Инварианты

Где возможно, выражайте правило через UNIQUE, CHECK, exclusion constraint или trigger. Ограничение в БД надёжнее одной прикладной проверки.

Повтор

При SQLSTATE 40001 повторяйте транзакцию целиком, а не только последний UPDATE.

Часть 3. Уровни изоляции — углублённо

3.1. Идея и синтаксис

Уровень изоляции — это **компромисс**: чем строже, тем меньше аномалий, но тем дороже (больше блокировок / выше шанс отката) и ниже параллелизм. Уровни **вложены**: более высокий запрещает всё, что запрещал более низкий. Стандарт разрешает СУБД выполнить транзакцию на уровне *строже* запрошенного (например, Read Uncommitted фактически как Read Committed).

Задаётся так (PostgreSQL):

```
-- на текущую транзакцию:
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
...
COMMIT;

-- или сразу в BEGIN:
BEGIN ISOLATION LEVEL SERIALIZABLE;

-- уровень по умолчанию для всех новых транзакций сессии:
SET SESSION CHARACTERISTICS AS TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Проверка текущего уровня

```
SHOW transaction_isolation;

BEGIN;
SELECT current_setting('transaction_isolation');
COMMIT;
```

Важно: SET TRANSACTION выполняется после BEGIN, но до первого запроса, который обращается к данным.

Практическая модель выбора

Сценарий	Начальная точка
Обычный короткий CRUD	Read Committed
Длинный согласованный отчёт	Repeatable Read
Проверка бизнес-инварианта по нескольким строкам	Serializable или явные блокировки
Очередь воркеров	Read Committed + FOR UPDATE SKIP LOCKED
Редкий конфликт по одной строке	Optimistic locking
Частые конфликты по одной строке	FOR UPDATE

3.2. Read Uncommitted

Формально допускает грязные чтения — транзакция видит незакоммиченные изменения других.

В PostgreSQL этого уровня фактически нет: запрос READ UNCOMMITTED молча повышается до READ COMMITTED. Причина — архитектурная: MVCC хранит версии строк, и механизма «показать чужую незакоммиченную версию» просто не существует. Грязные чтения невозможны by design.

Эксперимент: PostgreSQL повышает уровень

```
BEGIN ISOLATION LEVEL READ UNCOMMITTED;
SHOW transaction_isolation;
-- read uncommitted может отображаться как запрошенное имя,
-- но видимость данных соответствует Read Committed.
COMMIT;
```

Сессия А

```
BEGIN;
UPDATE accounts SET balance = 1 WHERE owner = 'Alice';
-- COMMIT пока не выполняем
```

Сессия В

```
BEGIN ISOLATION LEVEL READ UNCOMMITTED;
SELECT balance FROM accounts WHERE owner = 'Alice';
-- незакоммиченное значение 1 не видно
COMMIT;
```

Сессия А

Рекомендации

по решению проблем

Выбор изоляции

Начинайте с READ COMMITTED. Повышайте уровень только когда конкретная бизнес-операция требует стабильного снимка или защиты общего инварианта.

Настройка

Устанавливайте уровень после BEGIN, но до первого обращения к данным. Не меняйте глобальное значение без анализа всей нагрузки.

Очереди и конфликты

Очередям обычно нужен READ COMMITTED + FOR UPDATE SKIP LOCKED. Редкие конфликты по одной строке удобнее решать version-полем.


```
ROLLBACK;
```

3.3. Read Committed (по умолчанию)

Гарантия: **видишь только зафиксированные данные** (грязных чтений нет). Но повторяемость не гарантируется.

Ключевая деталь механики PostgreSQL: на этом уровне снимок берётся **на начало каждого оператора**, а не транзакции. Поэтому два `SELECT` в одной транзакции могут вернуть разное (`non-repeatable read`), а между ними может «влезть» чужой закоммиченный `INSERT` (`phantom`).

Тонкость с UPDATE. Если оператор `UPDATE` натывается на строку, которую параллельная транзакция уже изменила и закоммитила, PostgreSQL **не берёт старый снимок** для этой строки, а перечитывает её последнюю версию и применяет условие `WHERE` заново (`re-evaluation`). Это уменьшает потерянные обновления, но может дать неожиданное поведение при сложных условиях.

Когда использовать: подавляющее большинство OLTP-нагрузок, где нужна свежесть данных и не критична строгая повторяемость внутри транзакции.

Пример 1: новый снимок на каждый оператор

Эксперимент с `non-repeatable read` из Части 2 показывает основную механику: первый и второй `SELECT` получают разные снимки.

Можно увидеть идентификатор снимка:

```
BEGIN ISOLATION LEVEL READ COMMITTED;
SELECT pg_current_snapshot();
SELECT pg_sleep(3);
SELECT pg_current_snapshot();
COMMIT;
```

Содержимое снимка может отличаться, если параллельно фиксируются транзакции.

Пример 2: условное списание без отдельного SELECT

Плохой вариант:

```
SELECT balance FROM accounts WHERE id = 1;
-- проверка в приложении
UPDATE accounts SET balance = :newBalance WHERE id = 1;
```

Лучше объединить проверку и изменение:

```
UPDATE accounts
SET balance = balance - 100
WHERE id = 1
  AND balance >= 100
RETURNING balance;
```

Если возвращено 0 строк, денег недостаточно или счёт не найден. Между отдельной проверкой и записью нет окна гонки.

Пример 3: re-evaluation условия UPDATE

Сессия А

```
BEGIN;
UPDATE accounts
SET balance = balance - 900
WHERE owner = 'Alice' AND balance >= 900;
-- удерживаем блокировку, COMMIT пока не делаем
```

Сессия В

```
BEGIN ISOLATION LEVEL READ COMMITTED;
UPDATE accounts
SET balance = balance - 200
WHERE owner = 'Alice' AND balance >= 200;
-- запрос ждёт завершения А
```

После `COMMIT` сессии А сессия В перечитает новую версию строки и снова проверит `balance >= 200`. Если после первого списания осталось 100, строка перестанет удовлетворять условию, и В обновит 0 строк.

Когда Read Committed недостаточно

Если решение зависит от нескольких чтений или нескольких строк, каждый следующий запрос может увидеть уже другую картину. Тогда нужны:

- один атомарный SQL-оператор;
- явные row locks;

Рекомендации

по решению проблем

Окно гонки

Объединяйте проверку и изменение в одном операторе: `UPDATE ... WHERE` условие `RETURNING`. Ноль строк означает, что условие больше не выполнено.

Несколько чтений

Если решение зависит от нескольких запросов или строк, используйте `FOR UPDATE`, `REPEATABLE READ` или `SERIALIZABLE`.

Контроль результата

Всегда проверяйте `affected rows/RETURNING`. Не считайте успешным запрос, который обновил 0 строк.

Repeatable Read;
Serializable.

3.4. Repeatable Read

Гарантия по стандарту: повторное чтение той же строки даёт тот же результат (нет non-repeatable reads), но фантомы формально возможны.

PostgreSQL строже стандарта. Его Repeatable Read реализован через **snapshot isolation**: снимок фиксируется **один раз на старте транзакции**, и вся транзакция видит согласованную «фотографию» БД на этот момент. Следствия: - Нет non-repeatable reads И нет phantom reads — снимок неизменен. - Но **остаётся write skew** и другие аномалии сериализации, не покрытые тремя классическими феноменами.

Конфликты записи. Если транзакция на Repeatable Read пытается изменить строку, которую другая транзакция изменила и закоммитила после старта нашего снимка, PostgreSQL выдаёт ошибку:

```
ERROR: could not serialize access due to concurrent update
```

Транзакцию нужно **повторить целиком** (снять новый снимок). Приложение должно уметь ловить это и делать retry.

Когда использовать: отчёты и аналитика, где нужна согласованная картина на один момент; логика «прочитал набор — принял решение — записал», не требующая полной сериализуемости.

Пример 1: стабильный снимок

Сессия А

```
BEGIN ISOLATION LEVEL REPEATABLE READ;  
SELECT balance FROM accounts WHERE owner = 'Alice'; -- 1000
```

Сессия В

```
UPDATE accounts SET balance = 800 WHERE owner = 'Alice';
```

Сессия А

```
SELECT balance FROM accounts WHERE owner = 'Alice'; -- всё ещё 1000  
COMMIT;
```

После завершения транзакции новый запрос уже увидит 800.

Пример 2: конфликт конкурентного UPDATE

Сессия А

```
BEGIN ISOLATION LEVEL REPEATABLE READ;  
SELECT balance FROM accounts WHERE owner = 'Alice';
```

Сессия В

```
UPDATE accounts SET balance = balance - 100 WHERE owner = 'Alice';
```

Сессия А

```
UPDATE accounts SET balance = balance - 50 WHERE owner = 'Alice';  
-- ERROR: could not serialize access due to concurrent update  
ROLLBACK;
```

Нельзя повторить только последний UPDATE: вся логика транзакции строилась на старом снимке, поэтому повторяется транзакция целиком.

Пример 3: write skew сохраняется

Сначала верните обоих врачей в дежурство:

```
UPDATE doctors SET on_call = true;
```

Сессия А

```
BEGIN ISOLATION LEVEL REPEATABLE READ;  
SELECT count(*) FROM doctors WHERE on_call; -- 2  
UPDATE doctors SET on_call = false WHERE name = 'Иван';
```

Сессия В

```
BEGIN ISOLATION LEVEL REPEATABLE READ;  
SELECT count(*) FROM doctors WHERE on_call; -- 2  
UPDATE doctors SET on_call = false WHERE name = 'Мария';  
COMMIT;
```

Сессия А

```
COMMIT;
```

Рекомендации

по решению проблем

Ошибка 40001

Ловите serialization failure на уровне транзакционного сервиса. Делайте ограниченное число повторов с небольшим случайным backoff.

Старый snapshot

Повторяйте всю бизнес-операцию. Частичный повтор может использовать решения, принятые по уже устаревшим данным.

Write skew

REPEATABLE READ не защищает общий инвариант по разным строкам. Для него нужен SERIALIZABLE или согласованная явная блокировка набора.

Обе транзакции могут успешно завершиться, потому что изменяли разные строки. Проверка общего инварианта оказалась недостаточной.

3.5. Serializable — сильнейший

Гарантия: результат эквивалентен **какому-то** последовательному выполнению транзакций. Никаких аномалий, включая write skew.

Две реализации: - **Блокировочная (2PL + range locks)**: read/write-блокировки держатся до конца транзакции, плюс берутся **диапазонные блокировки** на предикаты WHERE, чтобы никто не вставил строку в уже прочитанный диапазон (защита от фантомов). Дорого, много ожиданий, риск дедлоков. - **SSI в PostgreSQL (Serializable Snapshot Isolation)**: работает поверх snapshot isolation без тяжёлых блокировок. СУБД отслеживает «опасные структуры» — циклы зависимостей чтение-запись между транзакциями (rw-конфликты). Если обнаруживается потенциально несерIALIZУемое пересечение, одна из транзакций откатывается:

```
ERROR: could not serialize access due to read/write dependencies among transactions
```

Практический паттерн retry (обязателен на Serializable и желателен на Repeatable Read):

```
while True:
    try:
        with db.transaction(isolation="serializable"):
            do_work()
        break
    except SerializationFailure:
        continue # снять новый снимок и повторить
```

Важно: транзакции на Serializable **должны быть идемпотентны при повторе** и по возможности короткими — чем длиннее, тем выше шанс конфликта. Для read-only транзакций накладные расходы SSI минимальны (можно пометить READ ONLY DEFERRABLE, чтобы гарантированно избежать откатов).

Эксперимент: Serializable ломает опасный цикл

Верните исходное состояние:

```
UPDATE doctors SET on_call = true;
```

Повторите сценарий write skew, но обе транзакции начните так:

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
```

Одна транзакция завершится, а другая с высокой вероятностью получит SQLSTATE 40001 и должна быть повторена. После повтора она увидит, что остался один дежурный, и не должна выключать второго.

Retry на JDBC/Kotlin

```
import java.sql.Connection
import java.sql.SQLException

fun <T> Connection.inSerializableTransaction(
    maxAttempts: Int = 4,
    block: (Connection) -> T
): T {
    val oldAutoCommit = autoCommit
    val oldIsolation = transactionIsolation

    try {
        repeat(maxAttempts) { attempt ->
            autoCommit = false
            transactionIsolation = Connection.TRANSACTION_SERIALIZABLE

            try {
                val result = block(this)
                commit()
                return result
            } catch (e: SQLException) {
                rollback()

                val retryable = e.sqlState == "40001" || // serialization_failure
                    e.sqlState == "40P01" // deadlock_detected
                if (!retryable || attempt == maxAttempts - 1) throw e
            }
        }
        error("unreachable")
    } finally {
        transactionIsolation = oldIsolation
        autoCommit = oldAutoCommit
    }
}
```

Практические правила retry:

Рекомендации

по решению проблем

Retry в проде

Обычно достаточно 3-5 попыток с jitter/backoff. После исчерпания попыток возвращайте контролируруемую ошибку, а не бесконечный цикл.

Внешние эффекты

Не выполняйте внутри повторяемой транзакции неидемпотентные HTTP-вызовы и отправку сообщений. Используйте idempotency key, outbox или эффект после COMMIT.

Длительность

Держите SERIALIZABLE-транзакции короткими. Чем меньше чтений, пауз и затронутых строк, тем меньше вероятность 40001.

повторяется вся транзакция, а не один запрос;
транзакция должна быть короткой;
внешние HTTP-вызовы и отправку сообщений нельзя бездумно повторять;
для внешних эффектов применяют idempotency key, transactional outbox или выполняют эффект после коммит;
полезен небольшой случайный backoff, чтобы конфликтующие транзакции не столкнулись сразу снова.

Read-only отчёт без риска serialization failure

```
BEGIN ISOLATION LEVEL SERIALIZABLE READ ONLY DEFERRABLE;  
SELECT ...; -- PostgreSQL может подождать безопасный снимок перед началом работы  
COMMIT;
```

Это подходит для длинного отчёта, которому важна сериализуемая согласованность, но не нужна запись.

Рекомендации

по решению проблем

Полный повтор

Повторяется вся транзакция вместе со всеми проверками и вычислениями.

Защита от дублей

Для внешних действий используйте уникальный idempotency key или transactional outbox. Иначе повтор после 40001/40P01 может создать дубликат.

Read-only отчёт

Для длинного строгого отчёта используйте SERIALIZABLE READ ONLY DEFERRABLE: PostgreSQL дождётся безопасного снимка до начала чтения.

3.6. Сводная таблица

Уровень	Dirty read	Non-repeatable	Phantom	Write skew
Read Uncommitted (ANSI)	да	да	да	да
Read Committed	нет	да	да	да
Repeatable Read (ANSI)	нет	нет	да	да
Repeatable Read (PostgreSQL)	нет	нет	нет	да
Serializable	нет	нет	нет	нет

«Serializable по отсутствию трёх феноменов» ≠ настоящая сериализуемость: отсутствие dirty/non-repeatable/phantom — необходимое, но не достаточное условие. Write skew проскакивает через них.

Мини-тест для самопроверки

В транзакции дважды прочитали одну строку и получили разные значения. Это **non-repeatable read**.

Дважды выполнили `count(*) WHERE status='new'`, и количество изменилось. Это **phantom read**.

Две транзакции изменили разные строки и нарушили общее правило. Это **write skew**.

PostgreSQL откатил транзакцию с 40001. Это не “поломка БД”, а штатный механизм обеспечения изоляции.

Быстрый выбор уровня

Нужен каждый запрос со свежими данными?
Нужна фотография БД на всю транзакцию?
Нужно гарантировать сложный общий инвариант?
Можно выразить операцию одним UPDATE/INSERT?

-> Read Committed
-> Repeatable Read
-> Serializable
-> часто Read Committed достаточно

Рекомендации

по решению проблем

Практический выбор

Свежесть каждого запроса - READ COMMITTED. Один стабильный снимок - REPEATABLE READ. Общий инвариант по нескольким строкам - SERIALIZABLE.

Не усложняйте

Если операция выражается одним атомарным INSERT/UPDATE с условием, READ COMMITTED часто достаточно.

40001

Это штатная часть протокола сериализуемости. Ошибкой приложения является отсутствие retry, а не сам откат.

Часть 4. Блокировки, MVCC и дедлоки — углублённо

4.1. Двухфазная блокировка (2PL)

Перед доступом к объекту транзакция берёт блокировку. «Двухфазная» = две фазы: **растущая** (только захват блокировок) и **сжимающая** (только освобождение). Как только отпустили первую блокировку — захватывать новые нельзя.

Strict 2PL — все *эксклюзивные* (write) блокировки держатся до конца транзакции (commit/rollback). Гарантирует восстанавливаемость (recoverability): никто не прочитает данные, которые потом откатятся.

2PL даёт сериализуемость, но снижает параллелизм (транзакции ждут) и порождает дедлоки.

Восстанавливаемость важнее сериализуемости. Сериализуемость можно ослабить ради производительности (более низкий уровень изоляции), а восстанавливаемость — нет: её нарушение ведёт к нарушению целостности данных.

Интуитивная схема

растущая фаза:	lock A -> lock B -> lock C
сжимающая фаза:	unlock C -> unlock B -> unlock A

В strict 2PL write-блокировки освобождаются только при commit или rollback. Поэтому другая транзакция не сможет прочитать незакоммиченное изменение через обычную блокировочную модель.

Почему блокировки уменьшают параллелизм

Если 100 транзакций хотят изменить одну строку счёта, они фактически выстраиваются в очередь. Индекс ускорит поиск строки, но не устранил логический конфликт за один ресурс.

4.2. Типы и режимы блокировок в PostgreSQL

По гранулярности: - **Блокировки уровня строки (row-level)** — берутся при UPDATE/DELETE/SELECT ... FOR UPDATE. Не мешают чтению (читатели идут по MVCC-снимку), мешают конкурирующей записи той же строки. - **Блокировки уровня таблицы (table-level)** — берутся операциями вроде ALTER TABLE, DROP, TRUNCATE, VACUUM FULL, а также неявно обычными запросами (в слабых режимах).

Основные строковые режимы (от слабого к сильному): - FOR KEY SHARE — слабейший; защищает ключевые столбцы от изменения/удаления (нужно для FK). - FOR SHARE — разделяемая; несколько транзакций могут держать её одновременно, но пишущий будет ждать. - FOR NO KEY UPDATE — как UPDATE, но не трогает ключи. - FOR UPDATE — эксклюзивная на строку; классика против lost update: «читаю для изменения — блокирую».

Совместимость (упрощённо): SHARE-режимы совместимы между собой, но не с эксклюзивными; эксклюзивные не совместимы ни с чем. Именно матрица совместимости определяет, кто кого будет ждать.

Advisory locks (рекомендательные) — блокировки «по договорённости», не привязанные к строкам/таблицам, а к числовому ключу: pg_advisory_lock(id). Полезны для прикладной сериализации (например, «одновременно только один воркер обрабатывает пользователя N»). СУБД их не проверяет автоматически — соблюдать должно приложение.

Пример: пессимистическая блокировка счёта

Сессия А

```
BEGIN;
SELECT id, balance
FROM accounts
WHERE owner = 'Alice'
FOR UPDATE;
```

Сессия В

```
UPDATE accounts SET balance = balance - 10 WHERE owner = 'Alice';
-- ждёт завершения А
```

Сессия А

```
UPDATE accounts SET balance = balance - 100 WHERE owner = 'Alice';
COMMIT;
```

После этого В продолжит работу уже с новой версией строки.

NOWAIT: не ждать блокировку

```
SELECT *
FROM accounts
```

Рекомендации

по решению проблем

Долгое ожидание

Не держите row lock во время пользовательского ввода, сетевых вызовов или тяжёлых вычислений. Сначала подготовьте данные, затем быстро откройте транзакцию.

Гранулярность

Блокируйте только нужные строки и только подходящим режимом. Слишком широкие locks снижают параллелизм и повышают риск дедлоков.

Advisory lock

Задайте единое правило преобразования бизнес-объекта в ключ и используйте его во всех сервисах. Иначе защита будет частичной.

```
WHERE owner = 'Alice'
FOR UPDATE NOWAIT;
```

Если строка занята, PostgreSQL сразу вернёт ошибку. Это полезно, когда приложение предпочитает быстрый отказ ожиданию.

SKIP LOCKED: очередь параллельных воркеров

```
CREATE TABLE jobs (
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  status      text NOT NULL DEFAULT 'queued',
  payload     jsonb NOT NULL
);
```

Атомарно забрать одну свободную задачу:

```
WITH picked AS (
  SELECT id
  FROM jobs
  WHERE status = 'queued'
  ORDER BY id
  FOR UPDATE SKIP LOCKED
  LIMIT 1
)
UPDATE jobs AS j
SET status = 'processing'
FROM picked
WHERE j.id = picked.id
RETURNING j.*;
```

Несколько воркеров не ждут одну занятую строку, а берут следующие. SKIP LOCKED нельзя использовать для обычных пользовательских выборов, где нужен полный согласованный набор: он намеренно пропускает занятые строки.

Advisory lock: прикладной mutex

```
SELECT pg_advisory_xact_lock(42); -- действует до конца транзакции
```

Например, ключ 42 может означать user_id=42. Все участники должны использовать одинаковое правило преобразования бизнес-объекта в ключ.

Диагностика ожиданий

```
SELECT
  pid,
  locktype,
  relation::regclass,
  mode,
  granted,
  pg_blocking_pids(pid) AS blocked_by
FROM pg_locks
JOIN pg_stat_activity USING (pid)
WHERE datname = current_database();
```

granted = false означает, что процесс ждёт блокировку.

4.3. MVCC изнутри

Вместо блокировки читателей PostgreSQL хранит **несколько версий строки** (кортежей, tuples). У каждого кортежа есть служебные поля: - **xmin** — id транзакции, которая создала эту версию. - **xmax** — id транзакции, которая удалила/заменила эту версию (0, если версия живая).

Видимость. Транзакция при чтении сравнивает свой снимок (список активных на его момент транзакций) с xmin/xmax кортежа и решает, видна ли ей эта версия. Грубо: версия видна, если создавшая транзакция уже закоммичена до нашего снимка, а удаляющая — ещё нет (или откатилась). Отсюда: - **Грязных чтений не бывает** — незакоммиченные версии просто не проходят проверку видимости. - **Читатели не блокируют писателей и наоборот** — они работают с разными версиями.

UPDATE — это не изменение на месте. PostgreSQL помечает старую версию (проставляет xmax) и создаёт новую версию строки. DELETE — только проставляет xmax. Значит, старые версии физически остаются в таблице.

Мёртвые кортежи и VACUUM. Версии, которые уже не видны ни одной активной транзакции, — «мёртвые». Их вычищает **VACUUM / autovacuum**, освобождая место под переиспользование и обновляя карту видимости. Без вакуума таблица и индексы «распухают» (**bloat**), запросы замедляются. Также вакуум решает проблему **wraparound** счётчика транзакций (xid 32-битный).

НОТ-обновления (Heap-Only Tuples). Если UPDATE не меняет проиндексированные столбцы и новая версия помещается на той же странице, PostgreSQL создаёт версию **без обновления индексов** (индекс продолжает

Рекомендации

по решению проблем

NOWAIT

Используйте, когда лучше быстро вернуть 'ресурс занят', чем держать запрос в ожидании. Ошибку нужно явно обработать в приложении.

SKIP LOCKED

Подходит только очередям и воркерам. Не применяйте для обычных пользовательских выборов, где нужен полный набор строк.

Диагностика

При зависании смотрите pg_locks, pg_stat_activity и pg_blocking_pids(). Не ограничивайтесь увеличением таймаута.

указывать на цепочку версий на странице). Это сильно удешевляет частые обновления неиндексированных полей — практический аргумент индексировать только то, что реально нужно.

Цена MVCC: мёртвые версии, необходимость вакуума, разрастание. Плата за то, что читатели никогда не ждут писателей.

Эксперимент: посмотреть версии строки

```
SELECT ctid, xmin::text, xmax::text, id, owner, balance
FROM accounts
WHERE owner = 'Alice';
```

Теперь выполните:

```
UPDATE accounts SET balance = balance + 1 WHERE owner = 'Alice';

SELECT ctid, xmin::text, xmax::text, id, owner, balance
FROM accounts
WHERE owner = 'Alice';
```

Обычный SELECT показывает только видимую версию. ctid и xmin обычно изменятся, потому что UPDATE создал новый tuple. Старую физическую версию можно исследовать расширениями вроде pageinspect, но для прикладного кода системные поля использовать нельзя.

Как читать xmin/xmax на уровне интуиции

```
xmin = транзакция, создавшая эту версию
xmax = транзакция, удалившая/заменившая или заблокировавшая версию
```

Реальная проверка видимости сложнее простого сравнения чисел: учитываются статус транзакций, снимок, subtransaction и MultiXact. На собеседовании важно не превращать xmax = 0 в абсолютное правило для всех случаев.

НОТ-обновление

Создадим индекс только по owner:

```
CREATE INDEX IF NOT EXISTS accounts_owner_idx ON accounts(owner);
```

Обновление balance не меняет индексированный столбец:

```
UPDATE accounts SET balance = balance + 1 WHERE owner = 'Alice';
```

Если новая версия помещается на той же heap-странице, PostgreSQL может сделать HOT update без новой записи во всех обычных индексах.

Статистика:

```
SELECT relname, n_tup_upd, n_tup_hot_upd
FROM pg_stat_user_tables
WHERE relname = 'accounts';
```

Теперь добавьте индекс по balance:

```
CREATE INDEX accounts_balance_idx ON accounts(balance);
```

Изменение balance уже обязано обновлять этот индекс, поэтому НОТ для таких обновлений станет невозможен. Это практическая цена “индексировать всё”.

Длинная транзакция мешает VACUUM

Если одна сессия долго держит старый снимок, старые версии могут оставаться потенциально видимыми этой транзакции. VACUUM не имеет права удалить их. Поэтому “idle in transaction” опаснее обычного idle-соединения.

Диагностика:

```
SELECT pid, state, xact_start, now() - xact_start AS age, query
FROM pg_stat_activity
WHERE xact_start IS NOT NULL
ORDER BY xact_start;
```

4.4. Дедлоки

Дедлок — транзакции ждут ресурсы по кругу:

```
T1 держит A, хочет B
T2 держит B, хочет A    → цикл, обе ждут вечно
```

Детектирование. PostgreSQL не проверяет цикл при каждом ожидании (дорого). Транзакция, заблокированная дольше deadlock_timeout (по умолчанию 1 с), запускает проверку графа ожиданий. Если найден цикл — одна транзакция (жертва) откатывается:

```
ERROR: deadlock detected
```

Приложение должно её повторить.

Рекомендации

по решению проблем

Idle in transaction

Настройте таймаут idle_in_transaction_session_timeout и мониторинг долгих xact_start. Такие сессии удерживают старый snapshot и мешают VACUUM.

НОТ и индексы

Не индексируйте часто меняющиеся столбцы без необходимости. Лишний индекс отключает НОТ для их обновлений и увеличивает bloat.

Deadlock

При 40P01 откатывайте и повторяйте транзакцию целиком. Одновременно исправляйте порядок доступа, иначе повторы лишь маскируют проблему.

Почему выше изоляция → выше риск дедлока: сильнее уровень → больше и «шире» блокировки (вплоть до диапазонных) → больше пересечений → выше шанс замкнуть цикл.

Как снижать вероятность: - **Единый порядок доступа** к объектам во всех транзакциях — тогда цикл не замкнётся (главный приём). - Короткие транзакции, быстрый commit. - Захватывать нужные блокировки заранее (SELECT ... FOR UPDATE), а не наращивать по ходу. - Понижать уровень изоляции там, где строгость не нужна. - Уменьшать гранулярность конфликтов (не блокировать лишнее).

Воспроизводимый дедлок

Верните балансы в любое корректное состояние.

Сессия А

```
BEGIN;
UPDATE accounts SET balance = balance + 1 WHERE owner = 'Alice';
```

Сессия В

```
BEGIN;
UPDATE accounts SET balance = balance + 1 WHERE owner = 'Bob';
```

Сессия А

```
UPDATE accounts SET balance = balance + 1 WHERE owner = 'Bob';
-- ждёт В
```

Сессия В

```
UPDATE accounts SET balance = balance + 1 WHERE owner = 'Alice';
-- замыкает цикл; одна транзакция будет отменена с 40P01
```

Исправление: единый порядок

Обе транзакции должны всегда блокировать счета, например, по возрастанию id:

```
SELECT id, balance
FROM accounts
WHERE id IN (:fromId, :toId)
ORDER BY id
FOR UPDATE;
```

После захвата обеих блокировок выполняются изменения. При одинаковом порядке ожидание возможно, но цикл не возникает.

Таймауты как страховка

```
SET LOCAL lock_timeout = '2s';
SET LOCAL statement_timeout = '5s';
```

lock_timeout не лечит архитектурную причину дедлока, но ограничивает бесконечное ожидание обычной блокировки.

4.5. Пессимистичный vs оптимистичный подход

Пессимистичный: заранее беру блокировки, считая конфликт вероятным (FOR UPDATE, 2PL). Меньше откатов, больше ожиданий/дедлоков. Хорош при высокой конкуренции за одни строки.

Оптимистичный: конфликт считаю редким, блокировок не беру, а на commit проверяю, не поменялось ли (версионный столбец или Serializable/Repeatable Read с retry). Меньше ожиданий, но нужны повторы. Хорош при низкой конкуренции.

Пессимистичный вариант

```
BEGIN;
SELECT id, balance FROM accounts WHERE id = 1 FOR UPDATE;
-- вычисления
UPDATE accounts SET balance = 900 WHERE id = 1;
COMMIT;
```

Подходит, если конфликт вероятен и операция короткая.

Оптимистичный вариант с version

Приложение читает id, balance, version.

Пользователь или сервис формирует новое значение.

Обновление выполняется только если версия не изменилась.

```
UPDATE accounts
SET balance = 900,
```

Рекомендации

по решению проблем

Главная профилактика

Во всех путях кода захватывайте одинаковые ресурсы в одном порядке, например по возрастанию id. Это разрывает возможность цикла ожидания.

Короткие транзакции

Заранее получите нужные идентификаторы, затем быстро захватите locks, выполните изменения и COMMIT.

Таймауты

lock_timeout и statement_timeout ограничивают ущерб, но не устраняют неверный порядок блокировок.

```
version = version + 1
WHERE id = 1
AND version = 7;
```

Если обновлено 0 строк, кто-то уже изменил запись. Приложение перечитывает данные и решает, повторять ли действие.

Kotlin-псевдокод

```
val updated = jdbc.update(
    """
    UPDATE accounts
    SET balance = ?, version = version + 1
    WHERE id = ? AND version = ?
    """.trimIndent(),
    newBalance, id, expectedVersion
)

if (updated == 0) {
    throw OptimisticLockException("Account was changed concurrently")
}
```

Как выбрать

Признак	Пессимистичный	Оптимистичный
Конфликт частый	Хорошо	Много повторов
Конфликт редкий	Лишнее ожидание	Хорошо
Операция долгая, ждёт пользователя	Плохо: долго держит lock	Лучше
Нужно гарантированно забрать задачу сейчас	Хорошо	Сложнее

Рекомендации

по решению проблем

Редкие конфликты

Используйте optimistic locking: version в WHERE, проверка 0 updated rows и повтор/сообщение пользователю.

Частые конфликты

Используйте SELECT ... FOR UPDATE, если операция короткая и конфликт за одну строку ожидаем.

Долгая операция

Не держите пессимистическую блокировку, пока пользователь думает. Сохраните version и проверьте её при записи.

Часть 5. Индексы — углублённо

5.1. Зачем и что хранит индекс

Без индекса поиск по условию — **seq scan** (читаем всю таблицу, $O(n)$). Индекс — отдельная структура, находящая строки, не читая всё, часто $O(\log n)$. Плата: место + замедление записи (каждый `INSERT/UPDATE/DELETE` обновляет и индексы) + необходимость обслуживания.

Индекс хранит не строки, а **ключ** → **указатель (TID)**: TID = (номер блока, смещение внутри блока) — физический адрес кортежа в heap-таблице. Найдя ключ, СУБД по TID идёт в таблицу за строкой.

Базовый эксперимент: Seq Scan до индекса

```
DROP INDEX IF EXISTS orders_customer_idx;
ANALYZE orders;

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM orders
WHERE customer_id = 4242;
```

Типичный узел:

```
Seq Scan on orders
  Filter: (customer_id = 4242)
 Rows Removed by Filter: ...
```

Создадим индекс:

```
CREATE INDEX orders_customer_idx ON orders(customer_id);
ANALYZE orders;
```

Повторный план часто станет Bitmap Heap Scan или Index Scan - выбор зависит от количества совпадений и распределения данных.

Почему $O(\log n)$ не означает “индекс всегда быстрее”

К стоимости поиска ключа добавляются:

чтение страниц индекса;

переходы по TID в heap;

случайный I/O;

проверка MVCC-видимости;

чтение большого числа подходящих строк.

Если подходит половина таблицы, последовательное чтение обычно дешевле тысяч случайных переходов.

5.2. B-дерево изнутри

По умолчанию `CREATE INDEX` создаёт **B-tree** (точнее B+-дерево).

Устройство на диске. Индекс — набор страниц (обычно по 8 КБ), организованных в дерево: - **Корень** и **внутренние узлы** содержат разделяющие ключи и указатели на нижележащие страницы. Служат «дорожными указателями». - **Листья** содержат сами ключи и указатели на строки (TID). В B+-дереве **все данные — только в листьях**. - Дерево **сбалансировано**: путь от корня до любого листа одинаков. Глубина $\approx \log(N)$, обычно 3–5 уровней даже на сотнях миллионов строк → максимум несколько чтений страниц на поиск. - Листья связаны в **двусвязный список** → быстрый проход по диапазону и в обе стороны (для `ORDER BY ASC/DESC`).

Поиск WHERE x = 42: 1. Корень → бинарный поиск по отсортированным ключам страницы → выбираем ветвь. 2. Спускаемся уровень за уровнем, каждый раз отбрасывая большую часть дерева. 3. В листе находим 42 и TID → идём в таблицу. Это $O(\log n)$.

Вставка и расщепление страниц (page split). При вставке ключ идёт в нужный лист. Если лист переполнен — он **расщепляется** на два, а разделяющий ключ поднимается в родителя (тот тоже может расщепиться, вплоть до роста дерева на уровень). Отсюда: - **fillfactor** — насколько плотно заполнять страницы при создании (по умолчанию 90% для B-tree). Меньше плотность → меньше расщеплений при будущих вставках, но больше размер. - Частые вставки в случайные места → частые расщепления и фрагментация; вставки в конец (возрастающий ключ, как `serial/timestamp`) → эффективно, расщепления только справа.

Почему LIKE 'foo%' использует индекс, а '%bar' — нет (главный пример на понимание): - Индекс отсортирован по значению слева направо. Все строки на foo лежат **подряд** — это непрерывный диапазон [`'foo'`, `'for'`). Дерево быстро находит начало и идёт по листьям. Работают `col LIKE 'foo%', col ~ '^foo'`. - `LIKE '%bar'` ищет по окончанию — совпадения **размазаны по всему дереву**, диапазона нет, индекс бесполезен.

Рекомендации

по решению проблем

Индекс не используется

Сначала проверьте селективность и стоимость heap-переходов. Seq Scan может быть правильным, даже когда индекс существует.

Проверка

Создавайте индекс на реалистичном объёме, выполняйте `ANALYZE` и сравнивайте `EXPLAIN (ANALYZE, BUFFERS)` до и после.

Цена записи

Перед добавлением индекса оцените частоту `INSERT/UPDATE/DELETE` и объём WAL. Индекс ускоряет чтение ценой записи и места.

- Общее правило: **B-tree помогает, когда условие задаёт непрерывный отсортированный диапазон ключей** (равенство, границы, префикс). Суффикс, функция от столбца, ведущий % — диапазон разрушен.

Нюансы шаблонов: вне локали C для индексирования LIKE-шаблонов нужен класс операторов text_pattern_ops. ILIKE/~* индексируются, только если шаблон начинается не с буквенных (регистрозависимых) символов.

Равенство

```
CREATE INDEX IF NOT EXISTS orders_customer_idx ON orders(customer_id);

EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders WHERE customer_id = 4242;
```

B-tree быстро находит первый ключ 4242, затем проходит соседние равные ключи в листьях.

Диапазон

```
CREATE INDEX orders_created_idx ON orders(created_at);

EXPLAIN (ANALYZE, BUFFERS)
SELECT id, created_at, total
FROM orders
WHERE created_at >= now() - interval '1 day';
```

Все даты последнего дня образуют непрерывный участок сортировки.

ORDER BY + LIMIT

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, created_at
FROM orders
ORDER BY created_at DESC
LIMIT 20;
```

B-tree можно читать в обратном направлении. План может избежать отдельного Sort и остановиться после первых 20 строк.

LIKE: префикс и суффикс

```
CREATE INDEX orders_email_pattern_idx
ON orders (email text_pattern_ops);

EXPLAIN (ANALYZE, BUFFERS)
SELECT id FROM orders WHERE email LIKE 'user123%';
```

Префикс превращается в ограниченный диапазон ключей.

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT id FROM orders WHERE email LIKE '%@example.com';
```

Ведущий % не задаёт начало диапазона B-tree. Для поиска подстроки обычно применяют расширение pg_trgm и GIN/GiST trigram index.

Page split на уровне идеи

```
лист заполнен: [10, 20, 30, 40]
вставляем 25
после split:  [10, 20, 25]  [30, 40]
родитель получает новый разделяющий ключ
```

Случайные UUID/значения распределяют вставки по дереву; возрастающий ключ обычно пишет в правую часть. Но у возрастающего ключа может появиться “горячая” правая страница при очень высокой конкуренции, поэтому нет универсального лучшего ключа для всех нагрузок.

5.3. Способы сканирования по индексу

Планировщик выбирает не только «индекс или seq scan», но и как по индексу идти:

- **Index Scan** — идём по индексу, для каждого совпадения прыгаем в heap за строкой. Хорош при малой выборке (высокая селективность).
- **Index Only Scan** — если все нужные столбцы есть в самом индексе, **в heap идти не надо** (проверяется ещё карта видимости — visibility map). Очень быстро. Достигается **покрывающим индексом**:

```
CREATE INDEX idx ON t (a) INCLUDE (b, c); -- b,c лежат в листьях, но не в ключе
```

- **Bitmap Index Scan** — сначала по индексу(ам) строим битовую карту нужных страниц heap, потом читаем heap **по порядку страниц** (последовательно, а не прыжками). Выгоден при средней выборке и при **комбинации нескольких индексов** (WHERE a=1 AND b=2 — два bitmap’a объединяются AND/OR).

Рекомендации

по решению проблем

LIKE с ведущим %

Для поиска подстроки используйте pg_trgm + GIN/GiST. B-tree не может быстро найти суффикс без известной левой границы.

Функция над столбцом

Создайте expression index с тем же выражением, которое используется в WHERE, например lower(email).

Page split

При случайных вставках следите за bloat и write amplification. Fillfactor снижайте только после измерений на реальной нагрузке.

Когда планировщик вообще НЕ берёт индекс: если условие малоселективно (выбирается, скажем, >5–10% таблицы), прыжки по индексу в heap дороже, чем прочитать таблицу подряд — seq scan выигрывает. Поэтому индекс на столбце с малым числом различных значений (пол, флаг) часто бесполезен без дополнительных приёмов.

Index Scan

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders WHERE id = 1000;
```

Типичный план:

```
Index Scan using orders_pkey on orders
Index Cond: (id = 1000)
```

Индекс дал TID, затем PostgreSQL сходил в heap за полной строкой.

Index Only Scan и INCLUDE

```
CREATE INDEX orders_customer_created_cover_idx
ON orders(customer_id, created_at DESC)
INCLUDE (total, status);

VACUUM (ANALYZE) orders;

EXPLAIN (ANALYZE, BUFFERS)
SELECT created_at, total, status
FROM orders
WHERE customer_id = 4242
ORDER BY created_at DESC
LIMIT 20;
```

Ищите:

```
Index Only Scan using orders_customer_created_cover_idx
Index Cond: (customer_id = 4242)
Heap Fetches: 0
```

Heap Fetches: 0 возможен, когда нужные heap-страницы отмечены all-visible. После массовых обновлений heap fetches могут снова появиться до следующего VACUUM.

Bitmap Index Scan

Создадим два отдельных индекса:

```
CREATE INDEX orders_status_idx ON orders(status);
CREATE INDEX orders_total_idx ON orders(total);
ANALYZE orders;

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM orders
WHERE status = 'paid'
AND total BETWEEN 1000 AND 1200;
```

Возможный фрагмент:

```
Bitmap Heap Scan on orders
-> BitmapAnd
    -> Bitmap Index Scan on orders_status_idx
    -> Bitmap Index Scan on orders_total_idx
```

Планировщик объединяет результаты индексов, затем читает подходящие heap-страницы по порядку.

Seq Scan при низкой селективности

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders WHERE status IN ('new', 'paid', 'cancelled');
```

Условие выбирает почти все строки. Индекс не уменьшает работу, поэтому Seq Scan логичен.

Рекомендации

по решению проблем

Большая выборка

Не заставляйте планировщик использовать индекс. Если запрос возвращает значительную часть таблицы, Seq Scan обычно дешевле.

Index Only Scan

Добавляйте INCLUDE только для частых узких запросов. Поддерживайте autovacuum, иначе Heap Fetches снова вырастут.

Ошибочная оценка

Если rows сильно отличается от actual rows, обновите статистику и проверьте распределение/корреляцию данных.

Что читать в EXPLAIN ANALYZE

Поле	Вопрос
actual time	Где реально потрачено время?
actual rows и rows	Насколько ошиблась оценка кардинальности?
loops	Сколько раз выполнялся узел?
Buffers: shared hit/read	Данные были в памяти или читались с диска?
Rows Removed by Filter	Сколько лишних строк прочитано?
Heap Fetches	Насколько index-only scan действительно был “only”?

5.4. Составные и частичные индексы

Составной (multicolumn) индекс (a, b, c). Сортировка лексикографическая: сначала по a, при равенстве — по b, затем c. Отсюда **правило левого префикса**: - Работает для условий по a, (a, b), (a, b, c). - Работает для a = ... AND b > ... (равенства слева, диапазон — последним полезным столбцом). - **Не** работает для условия только по b или только по c — по той же причине, что суффикс не индексируется: пока не зафиксирован a, значения b не образуют непрерывный диапазон. - Порядок столбцов в индексе — проектное решение: столбцы с равенством ставят раньше, диапазонные — позже.

Частичный (partial) индекс — индексирует только часть строк по условию:

```
CREATE INDEX idx ON orders (created_at) WHERE status = 'active';
```

Меньше размер, быстрее, обслуживает только «горячие» строки. Идеален, когда запросы всегда фильтруют по одному предикату (например, только активные заказы).

Индекс по выражению (functional):

```
CREATE INDEX idx ON users (lower(email));
```

Нужен, чтобы WHERE lower(email) = ... использовал индекс — обычный индекс по email его не увидит, т.к. функция разрушает исходный порядок.

Уникальный индекс реализует ограничение UNIQUE/PRIMARY KEY и одновременно ускоряет поиск по ключу.

Составной индекс под конкретный запрос

```
CREATE INDEX orders_customer_created_idx ON orders(customer_id, created_at DESC);
```

Хорошо соответствует запросу:

```
SELECT * FROM orders WHERE customer_id = 4242 AND created_at >= now() - interval '30 days' ORDER BY created_at DESC;
```

Логика порядка:

customer_id = ... фиксирует левую группу.

Внутри группы даты уже отсортированы.

Диапазон и ORDER BY работают по второму столбцу.

Запрос только по второму столбцу

```
SELECT * FROM orders WHERE created_at >= now() - interval '1 hour';
```

Базовая модель: отдельный индекс по created_at обычно надёжнее, потому что без фиксированного customer_id нужные даты распределены по множеству групп.

Актуальное уточнение PostgreSQL 18: планировщик умеет применять B-tree skip scan в некоторых случаях и может использовать составной индекс даже без обычного условия на ведущий столбец, особенно если у него мало различных значений. Это оптимизация, а не отмена проектного правила: индекс всё равно проектируют под реальные ведущие условия и подтверждают решением EXPLAIN.

Частичный индекс

```
CREATE INDEX orders_new_created_idx ON orders(created_at DESC) WHERE status = 'new';
```

Используется запросом, чей предикат логически подразумевает условие индекса:

Рекомендации

по решению проблем

Порядок столбцов

Ставьте столбцы с равенством первыми, затем диапазон/ORDER BY. Проектируйте индекс под конкретную форму запроса.

Запрос по неведущему полю

Не рассчитывайте на skip scan как на основную стратегию. Для частого самостоятельного фильтра создайте отдельный индекс.

Partial index

Предикат запроса должен логически включать условие индекса. Параметризованный или иной предикат может не позволить планировщику доказать соответствие.

```
SELECT *
FROM orders
WHERE status = 'new'
ORDER BY created_at DESC
LIMIT 50;
```

Запрос без `status = 'new'` не может безопасно использовать этот индекс: в нём физически отсутствуют остальные строки.

Индекс по выражению

```
CREATE UNIQUE INDEX orders_email_lower_uq
ON orders(lower(email));

SELECT * FROM orders WHERE lower(email) = lower('USER42@EXAMPLE.COM');
```

Обычный индекс по `email` не упорядочен по результату `lower(email)`, поэтому выражение должно совпадать с индексным выражением.

Покрывающий индекс

```
CREATE INDEX orders_customer_cover_idx
ON orders(customer_id)
INCLUDE (status, total, created_at);
```

`INCLUDE`-столбцы не участвуют в сортировке и поиске. Они являются `payload` для `index-only scan`.

Цена широкого индекса

- больше размер и давление на `cache`;
- дороже `INSERT/UPDATE/DELETE`;
- больше риск превысить размер индексной записи;
- обновление `INCLUDE`-столбца также требует изменения индекса и мешает `HOT`.

Рекомендации

по решению проблем

Широкий индекс

Не добавляйте `INCLUDE` 'на всякий случай'. Каждый `payload`-столбец увеличивает размер, стоимость записи и может мешать `HOT`.

Уникальность email

Для регистронезависимой уникальности используйте `UNIQUE expression index` по `lower(email)` и одинаковую нормализацию в приложении.

Проверка пользы

Сравните размер индекса, `Heap Fetches` и `latency` ключевого запроса. Удаляйте покрытие, которое не даёт измеримого выигрыша.

5.5. Типы индексов PostgreSQL (что и когда)

Тип	Устройство	Операторы	Ниша
B-tree	сбалансированное отсортированное дерево	= < <= > >=, BETWEEN, IN, префикс LIKE, сортировка	90% случаев
Hash	32-битный хеш значения	только =	точное равенство, порядок не нужен
GiST	сбалансированное дерево + классы операторов	геометрия, &&, @>, диапазоны, k-NN	гео, range-типы, ближайший сосед, full-text
SP-GiST	несбалансированные разбиения (quad-, k-d, radix)	пространственные, префиксные	точки, IP-диапазоны, префиксы
GIN	инвертированный индекс (элемент → строки)	@> <@ && =	массивы, JSONB, full-text
BRIN	сводки min/max по зонам блоков	< <= = >= >	огромные таблицы с корреляцией порядка (даты, логи)

Синтаксис выбора типа: CREATE INDEX имя ON таблица USING GIN (столбец);

Детали по нестандартным типам: - **Hash** — только равенство; в старых версиях не журналировался (сейчас надёжен). Реальная ниша узкая — B-tree на равенстве почти так же хорош и умеет больше. - **GiST** — не индекс, а *инфраструктура*: что умеет, зависит от класса операторов. Фишка — **k-NN** (ORDER BY location <-> point '(x,y)' LIMIT k) для «ближайших соседей». - **SP-GiST** — для данных с естественным пространственным/иерархическим разбиением; несбалансированные структуры. - **GIN** — инвертированный: для каждого элемента (токен, ключ массива/JSON) хранит список строк. Отвечает на «в каких строках *есть* элемент X». Медленнее на вставку (можно ускорить fastupdate), но незаменим для JSONB и полнотекста. - **BRIN** — крошечный; хранит сводку по зонам блоков. Эффективен, **только если физический порядок строк коррелирует со значением** столбца (append-only лог по времени). Нет корреляции → бесполезен. - **bloom** (расширение) — на фильтрах Блума; когда фильтруют по многим столбцам в произвольных сочетаниях.

B-tree: обычные OLTP-запросы

```
CREATE INDEX ON orders(customer_id);
SELECT * FROM orders WHERE customer_id = 42;
```

Hash: только равенство

```
CREATE INDEX orders_email_hash_idx ON orders USING hash(email);
SELECT * FROM orders WHERE email = 'user42@example.com';
```

На практике B-tree часто предпочтительнее: также поддерживает равенство, но дополнительно умеет диапазоны и сортировку.

GIN для JSONB

```
CREATE INDEX orders_payload_gin_idx
ON orders USING gin(payload jsonb_path_ops);

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM orders
WHERE payload @> '{"channel":"mobile"}'::jsonb;
```

Инвертированный индекс хранит отдельные ключи/элементы и списки TID строк, где они встречаются. Он отвечает на вопрос “в каких документах есть элемент?”.

jsonb_path_ops обычно компактнее и эффективнее для @>, но поддерживает более узкий набор операций, чем стандартный jsonb_ops.

GIN для массива

```
CREATE TABLE articles (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  tags text[] NOT NULL
);

CREATE INDEX articles_tags_gin_idx ON articles USING gin(tags);

SELECT * FROM articles WHERE tags @> ARRAY['postgresql'];
```

GiST для пересечения диапазонов

```
CREATE TABLE reservations (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

Рекомендации

по решению проблем

Тип по умолчанию

Начинайте с B-tree для равенства, диапазона и сортировки. Выбирайте другой тип только под конкретный оператор и структуру данных.

GIN

Используйте для JSONB, массивов и полнотекста, но учитывайте более дорогие вставки и обслуживание pending list.

BRIN

Проверяйте корреляцию столбца с физическим порядком. Без неё BRIN почти не исключает диапазоны страниц.

```
room_id bigint NOT NULL,
period tstzrange NOT NULL
);

CREATE INDEX reservations_period_gist_idx
ON reservations USING gist(period);

SELECT *
FROM reservations
WHERE period && tstzrange(
    '2026-08-10 10:00+00',
    '2026-08-10 12:00+00',
    '[')
);
```

Оператор && означает пересечение диапазонов.

BRIN для огромного append-only лога

```
CREATE TABLE events (
    id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    created_at  timestampz NOT NULL,
    payload     jsonb NOT NULL
);

CREATE INDEX events_created_brin_idx
ON events USING brin(created_at)
WITH (pages_per_range = 64);
```

BRIN не хранит каждый ключ. Он хранит сводку по диапазонам heap-страниц. Если записи добавляются примерно по времени, запрос за один день исключит большинство диапазонов. Если физический порядок случайный, почти каждый диапазон будет пересекаться с запросом, и индекс станет слабым.

Как выбрать тип по вопросу

```
"равно / больше / между / сортировка"      -> B-tree
"документ содержит ключ/слово/элемент"     -> GIN
"объекты пересекаются / ближайший сосед"   -> GiST
"огромная таблица, значение связано с порядком" -> BRIN
"только равно, порядок не нужен"           -> Hash, но сравнить с B-tree
```

5.6. Обслуживание индексов

Bloat — из-за MVCC индексы накапливают ссылки на мёртвые версии и распухают; лечится autovacuum, при сильном распухании — REINDEX (можно REINDEX ... CONCURRENTLY, не блокируя запись).

Создание без блокировки: CREATE INDEX CONCURRENTLY строит индекс, не блокируя запись в таблицу (дольше, но безопасно на проде).

Не индексируйте всё подряд: каждый индекс замедляет запись и мешает HOT-обновлениям. Индексируйте под реальные запросы; лишние индексы удаляйте (их использование видно в pg_stat_user_indexes).

Проверка плана: EXPLAIN (ANALYZE, BUFFERS) <запрос> показывает, использовался ли индекс, каким сканом и сколько строк реально прочитано. Основной инструмент диагностики.

Создание индекса на проде

```
CREATE INDEX CONCURRENTLY orders_created_prod_idx
ON orders(created_at);
```

Особенности:

команда дольше обычного CREATE INDEX;

не может выполняться внутри transaction block;

при сбое может остаться INVALID index, который нужно удалить или перестроить;

запись в таблицу не блокируется обычным длительным эксклюзивным lock, но краткие блокировки и нагрузка всё равно есть.

Проверка использования индексов

```
SELECT
    schemaname,
    relname,
    indexrelname,
    idx_scan,
    idx_tup_read,
    idx_tup_fetch
FROM pg_stat_user_indexes
ORDER BY idx_scan ASC, indexrelname;
```

Рекомендации

по решению проблем

Создание на проде

Используйте CREATE INDEX CONCURRENTLY, планируйте дополнительную нагрузку и после сбоя проверяйте indisvalid/indisready.

Bloat

Сначала подтвердите проблему статистикой и размером. REINDEX CONCURRENTLY применяйте при реальном bloat, а не по расписанию без измерений.

Лишние индексы

Сопоставляйте pg_stat_user_indexes с полным циклом нагрузки. Не удаляйте UNIQUE/FK или редкий критический индекс только из-за idx_scan = 0.

Нулевой `idx_scan` не всегда означает “удалить”: индекс мог быть новым, нужен редкому критическому запросу или обслуживать `UNIQUE/FK`-паттерн. Сначала сопоставляют статистику с периодом наблюдения и `workload`.

Размеры

```
SELECT
  pg_size_pretty(pg_relation_size('orders')) AS heap,
  pg_size_pretty(pg_indexes_size('orders')) AS indexes,
  pg_size_pretty(pg_total_relation_size('orders')) AS total;
```

REINDEX

```
REINDEX INDEX CONCURRENTLY orders_customer_idx;
```

Используют при подтверждённом сильном `bloat` или повреждении, а не как ежедневный ритуал.

VACUUM и ANALYZE - разные задачи

```
VACUUM (ANALYZE) orders;
```

`VACUUM` освобождает место мёртвых версий для повторного использования и обслуживает `visibility map/freeze`.

`ANALYZE` собирает статистику распределения для планировщика.

`VACUUM FULL` физически переписывает таблицу и требует тяжёлой блокировки; это не обычный `vacuum`.

Мини-чеклист перед созданием индекса

Какой конкретный запрос он ускоряет?

Каковы `equality`, `range` и `ORDER BY` условия?

Сколько строк возвращается?

Можно ли сделать `partial index`?

Нужен ли `INCLUDE` для `index-only scan`?

Как индекс повлияет на `write-heavy` нагрузку и `HOT`?

Подтвердил ли `EXPLAIN (ANALYZE, BUFFERS)` улучшение на реалистичных данных?

5.7. Ключевые интуиции (чтобы не зубрить)

Порядок ключей в структуре определяет, что она ускорит. Отсортированное дерево (B-tree) → равенство, диапазоны, префиксы, сортировка. Хеш → только равенство (порядок уничтожен). Инвертированный (GIN) → «содержит элемент».

Индекс помогает, когда условие сужается до непрерывного участка структуры. Как только совпадения «размазаны» (суффикс, только не-левый столбец составного индекса, функция без `functional`-индекса) — индекс не применить.

Индекс ускоряет чтение, но замедляет запись и ест место. Ставить осознанно.

Малоселективные условия индекс не спасают — планировщик выберет `seq scan`.

MVCC ↔ индексы: обновление неиндексированных полей дешевле (`HOT`), лишние индексы этому мешают.

Разбор типичных ошибок

Ошибка: “Есть индекс, значит PostgreSQL обязан его использовать”.

Правильно: планировщик выбирает самый дешёвый план по оценке. Индекс может быть дороже `Seq Scan`.

Ошибка: “Индекс по (a, b) полностью бесполезен для b”.

Правильно: базово он хуже подходит без `a`; в PostgreSQL 18 возможен `skip scan`, но решение зависит от статистики и стоимости.

Ошибка: “Index Only Scan никогда не читает таблицу”.

Правильно: при `unset all-visible bit` он делает `heap fetch` для проверки `MVCC`-видимости.

Ошибка: “VACUUM уменьшает файл таблицы”.

Правильно: обычный `VACUUM` освобождает место внутри файла для повторного использования. Возврат места ОС обычно требует переписывания (`VACUUM FULL`, `CLUSTER` или другие методы).

Ошибка: “Serializable означает отсутствие ошибок”.

Правильно: он обеспечивает корректный результат ценой штатных откатов и обязательного `retry`.

Что чаще всего спрашивают

Различие `dirty / non-repeatable / phantom + lost update / write skew`.

Рекомендации

по решению проблем

VACUUM vs ANALYZE

`VACUUM` обслуживает версии и `visibility map`; `ANALYZE` обновляет статистику планировщика. Для диагностики медленного плана часто нужны оба.

VACUUM FULL

Не используйте как обычное обслуживание: он переписывает таблицу и требует тяжёлой блокировки. Рассматривайте только при подтверждённой необходимости.

Перед индексом

Зафиксируйте конкретный запрос, ожидаемую селективность и влияние на запись. Улучшение подтверждайте `EXPLAIN` на реалистичных данных.

Какой уровень какие аномалии убирает; что PostgreSQL строже стандарта (RR без фантомов, но write skew остаётся).

Уровень по умолчанию — Read Committed. На Serializable/RR нужен retry при ошибке сериализации.

Блокировки (2PL, режимы, FOR UPDATE) vs MVCC (xmin/xmax, снимки, VACUUM, HOT).

Дедлок: цикл ожидания, детект по deadlock_timeout, лечится единым порядком доступа.

Почему LIKE 'foo%' по индексу, а '%bar' — нет; почему составной (a, b) не работает по одному b.

B-tree vs Hash vs GIN vs BRIN; index-only / bitmap scan; partial и covering (INCLUDE) индексы.

Рекомендации

по решению проблем

Как отвечать

На собеседовании сначала назовите проблему, затем механизм PostgreSQL и только после этого инструмент решения.

Главные связи

Lost update -> atomic UPDATE/locking.
Write skew -> SERIALIZABLE. Deadlock -> единый порядок + retry. Медленный запрос -> ANALYZE + EXPLAIN BUFFERS.

Не зубрить

Объясняйте через структуру: snapshot определяет видимость, lock - порядок записи, индекс - расположение ключей и стоимость heap-доступа.

Часть 6. Практические задания с решениями

Задание 1. Безопасный перевод между счетами

Условие: перевести сумму между двумя счетами. Не допустить отрицательного баланса и дедлока при встречных переводах.

Решение

```
BEGIN;

SELECT id, balance
FROM accounts
WHERE id IN (:from_id, :to_id)
ORDER BY id
FOR UPDATE;

UPDATE accounts
SET balance = balance - :amount
WHERE id = :from_id
  AND balance >= :amount;

-- приложение проверяет, что обновлена 1 строка

UPDATE accounts
SET balance = balance + :amount
WHERE id = :to_id;

COMMIT;
```

Почему работает:

- строки блокируются в едином порядке;
- проверка остатка и списание находятся в одном UPDATE;
- обе записи атомарны в одной транзакции;
- при любой ошибке выполняется ROLLBACK.

Задание 2. Выбрать уровень для отчёта

Условие: отчёт читает 20 таблиц 5 минут. Все показатели должны описывать один момент времени, запись не выполняется.

Решение

Обычно REPEATABLE READ READ ONLY достаточно для согласованного снимка. Если нужна строгая сериализуемость относительно бизнес-транзакций, использовать:

```
BEGIN ISOLATION LEVEL SERIALIZABLE READ ONLY DEFERRABLE;
-- отчёт
COMMIT;
```

Задание 3. Найти проблему индекса

Есть индекс:

```
CREATE INDEX idx_orders_status_created
ON orders(status, created_at DESC);
```

Частый запрос:

```
SELECT *
FROM orders
WHERE customer_id = 42
ORDER BY created_at DESC
LIMIT 20;
```

Решение

Текущий индекс не содержит customer_id и не помогает отфильтровать клиента. Подходящий вариант:

```
CREATE INDEX idx_orders_customer_created
ON orders(customer_id, created_at DESC);
```

Если запрос выбирает только несколько столбцов, можно добавить INCLUDE.

Рекомендации

по решению проблем

Перевод

После условного списания обязательно проверьте, что обновлена 1 строка. Если нет - ROLLBACK и бизнес-ошибка 'недостаточно средств/счёт не найден'.

Отчёт

Помечайте длинный отчёт READ ONLY. Для строгой сериализуемости используйте DEFERRABLE, чтобы получить безопасный snapshot до начала работы.

Индекс

Проверьте запрос с LIMIT на реальном распределении данных. INCLUDE добавляйте только если он действительно даёт Index Only Scan.

Задание 4. Почему индекс по status не используется

```
SELECT * FROM orders WHERE status = 'paid';
```

Третя таблицы имеет статус paid.

Решение

Низкая селективность: читать множество TID и heap-страниц дороже последовательного прохода. Возможные варианты:

принять Seq Scan;

сделать partial index для более узкого бизнес-запроса;

добавить более селективный ведущий столбец;

вернуть меньше столбцов и использовать covering index, если workload это оправдывает.

Задание 5. JSONB-фильтр

```
SELECT *
FROM orders
WHERE payload @> '{"country":"RU", "channel":"mobile"}';
```

Решение

```
CREATE INDEX orders_payload_path_idx
ON orders USING gin(payload jsonb_path_ops);
```

После создания выполнить ANALYZE и проверить план.

Задание 6. Исправить lost update

Плохой код:

```
SELECT balance
newBalance = balance - amount
UPDATE accounts SET balance = newBalance
```

Решение A: атомарный UPDATE

```
UPDATE accounts
SET balance = balance - :amount
WHERE id = :id AND balance >= :amount
RETURNING balance;
```

Решение B: optimistic locking

```
UPDATE accounts
SET balance = :new_balance,
    version = version + 1
WHERE id = :id AND version = :old_version;
```

Решение C: pessimistic locking

```
SELECT balance FROM accounts WHERE id = :id FOR UPDATE;
```

Выбор зависит от частоты конфликтов и длительности операции.

Задание 7. Устранить дедлок

Транзакция A меняет счета 1 -> 2, транзакция B меняет 2 -> 1.

Решение

Перед изменениями обе транзакции блокируют оба счета по одинаковому правилу, например ORDER BY id FOR UPDATE. Дополнительно приложение повторяет транзакцию при 40P01.

Задание 8. Спроектировать индекс для очереди

```
SELECT id
FROM jobs
WHERE status = 'queued'
ORDER BY id
FOR UPDATE SKIP LOCKED
LIMIT 1;
```

Рекомендации

по решению проблем

Низкая селективность

При status='paid' для трети таблицы примите Seq Scan либо добавьте более селективное условие/partial index под конкретный hot-path.

JSONB

После GIN выполните ANALYZE и сравните jsonb_path_ops с jsonb_ops по реально используемым операторам.

Конкурентные записи

Для lost update выбирайте один основной паттерн и стандартизируйте его в слое доступа к данным. Для дедлока добавьте retry 40P01 и единый lock order.

Решение

```
CREATE INDEX jobs_queued_id_idx
ON jobs(id)
WHERE status = 'queued';
```

Partial index содержит только ожидающие задачи и уже упорядочен по id.

Часть 7. Вопросы на собеседовании и короткие ответы

Транзакции и изоляция

1. Чем Atomicity отличается от Durability?

Atomicity отвечает за “всё или ничего” внутри транзакции. Durability - за сохранность уже подтверждённого коммит после сбоя.

2. Почему Consistency не обеспечивается только СУБД?

СУБД обеспечивает объявленные ограничения. Если бизнес-инвариант не выражен constraint, trigger, правильной изоляцией или кодом, она о нём не знает.

3. Чем non-repeatable read отличается от phantom read?

В первом меняется значение той же строки; во втором меняется состав множества, удовлетворяющего предикату.

4. Какой уровень по умолчанию в PostgreSQL?

Read Committed.

5. Почему в PostgreSQL нет dirty read?

MVCC не показывает незакоммиченные версии другим транзакциям; Read Uncommitted имеет поведение Read Committed.

6. Где берётся snapshot на Read Committed?

В начале каждого оператора.

7. Где берётся snapshot на Repeatable Read?

Один стабильный snapshot используется всей транзакцией.

8. Почему Repeatable Read не гарантирует сериализуемость?

Возможен write skew: транзакции читают общий набор, но изменяют разные строки.

9. Что делать с SQLSTATE 40001?

Повторить всю транзакцию с новым снимком.

10. Почему retry должен учитывать внешние эффекты?

HTTP-вызов или отправка сообщения могли уже произойти; без идемпотентности повтор создаст дубль.

Блокировки и MVCC

11. Блокирует ли обычный SELECT UPDATE?

Обычно нет: SELECT читает подходящую MVCC-версию. Исключения связаны с явными locks и тяжёлыми DDL locks.

12. Что блокирует SELECT FOR UPDATE?

Конкурирующие изменения/удаления и несовместимые row-lock режимы для выбранных строк.

13. Когда использовать SKIP LOCKED?

Для очередей и параллельных воркеров, где допустимо временно пропустить занятую задачу.

14. Что такое xmin и xmax?

Системные поля tuple: транзакция создания и транзакция удаления/замены/блокировки версии. Реальная семантика xmax может включать MultiXact.

15. Почему UPDATE создаёт новую версию?

MVCC позволяет старым снимкам продолжать видеть старую версию, не блокируя читателей.

16. Зачем VACUUM?

Убирать недоступные версии для повторного использования места, обслуживать visibility map и предотвращать XID wraparound.

17. Что такое HOT?

Оптимизация UPDATE без новых записей в обычных индексах, когда не меняются индексированные столбцы и новая версия помещается на той же странице.

18. Как избежать дедлока?

Единый порядок захвата ресурсов, короткие транзакции, нужный lock сразу и retry при 40P01.

Рекомендации

по решению проблем

Очередь jobs

Partial index должен соответствовать status='queued'. Обновление status удалит запись из индекса, поэтому следите за write-нагрузкой и autovacuum.

SKIP LOCKED

Забирайте и меняйте статус одной атомарной командой с RETURNING, чтобы два воркера не получили одну задачу.

Ответы

В коротком ответе всегда добавляйте одно практическое следствие: что обработать в приложении или какой SQL-паттерн применить.

Индексы

19. Что хранит B-tree index в PostgreSQL?

Отсортированные ключи и TID hear-версий; внутренние страницы направляют поиск, листья содержат индексные записи.

20. Почему LIKE 'foo%' может использовать B-tree?

Совпадения образуют непрерывный диапазон по левому префиксу.

21. Почему LIKE '%foo' обычно не может?

Совпадения разбросаны по всему порядку ключей; нет известной нижней границы диапазона.

22. Что такое селективность?

Доля строк, которую оставляет условие. Чем меньше подходящих строк, тем чаще выгоден индекс.

23. Чем Index Scan отличается от Bitmap Heap Scan?

Index Scan сразу прыгает к каждой строке; bitmap сначала группирует TID по heap-страницам и читает страницы упорядоченно.

24. Когда возможен Index Only Scan?

Все требуемые столбцы доступны в index, а visibility map позволяет часто не посещать heap.

25. Что делает INCLUDE?

Добавляет payload-столбцы в листья index для covering query, не включая их в поисковый ключ.

26. Что такое правило левого префикса?

Составной B-tree эффективнее всего ограничивается равенствами по ведущим столбцам и диапазоном по следующему. PostgreSQL 18 также может применять skip scan, но это cost-based исключение, а не основа дизайна.

27. Когда нужен partial index?

Когда большинство полезных запросов работает с устойчивым узким подмножеством строк, например только status='queued'.

28. Когда нужен expression index?

Когда условие использует функцию или выражение, например lower(email).

29. GIN против B-tree?

B-tree ищет положение целого отсортированного ключа; GIN ищет элементы внутри составного значения - JSONB, массива, документа.

30. Когда BRIN эффективен?

На огромной таблице, где значение коррелирует с физическим порядком строк, например append-only timestamp.

31. Почему лишний индекс вреден?

Занимает место, замедляет запись, увеличивает WAL и может мешать HOT update.

32. С чего начинать анализ медленного запроса?

С реалистичных данных, актуального ANALYZE и EXPLAIN (ANALYZE, BUFFERS), затем сравнить estimate/actual rows и объём I/O.

Рекомендации

по решению проблем

Про индексы

Не ограничивайтесь названием типа скана. Объясните, сколько heap-страниц читается, нужна ли проверка видимости и почему планировщик оценил этот путь дешевле.

Составной индекс

Говорите о лексикографическом порядке и непрерывном диапазоне, а skip scan упоминайте как cost-based оптимизацию PostgreSQL 18.

Медленный запрос

Начинайте с EXPLAIN (ANALYZE, BUFFERS), сравнения estimate/actual rows и I/O, а не с немедленного CREATE INDEX.

Часть 8. Итоговая шпаргалка

Изоляция

Read Committed:

- snapshot на каждый statement
- свежие committed данные
- возможны non-repeatable read и phantom

Repeatable Read в PostgreSQL:

- snapshot на транзакцию
- нет non-repeatable и phantom
- возможен write skew
- возможен 40001 при concurrent update

Serializable:

- эффект как у последовательного выполнения
- PostgreSQL SSI ищет опасные rw-зависимости
- 40001 является штатным результатом
- retry всей транзакции обязателен

Блокировки

FOR UPDATE -> читаю строку, потому что собираюсь менять
NOWAIT -> не ждать, сразу ошибка
SKIP LOCKED -> пропустить занятое, подходит очередям
advisory lock -> прикладная блокировка по договорённости

MVCC

UPDATE = старая tuple + новая tuple
xmin = создатель версии
xmax = удалившая/заменившая/заблокировавшая транзакция или MultiXact
VACUUM = очистка мёртвых версий + visibility/freeze
HOT = update без новых записей в обычных индексах при выполнении условий

Индексы

B-tree -> equality, range, prefix, ORDER BY
GIN -> element inside JSONB/array/text document
GiST -> ranges, geometry, nearest-neighbor infrastructure
BRIN -> summaries of page ranges for huge correlated tables

Equality columns first, range/order column next.
Verify with EXPLAIN; do not trust the mere existence of an index.

Рекомендации

по решению проблем

Операционный чек-лист

1) Короткая транзакция. 2) Проверка affected rows. 3) Retry 40001/40P01. 4) Идемпотентность внешних эффектов. 5) Единый порядок locks.

Индексный чек-лист

1) Реальный запрос. 2) ANALYZE. 3) EXPLAIN BUFFERS. 4) Селективность. 5) Влияние на запись/HOT. 6) Проверка после внедрения.

Мониторинг

Следите за долгими транзакциями, блокировками, autovacuum, bloat и ошибками сериализации. Корректность без наблюдаемости быстро деградирует.

Источники и актуальные уточнения

Теоретическая основа взята из исходного подробного конспекта пользователя. Примеры и актуальные замечания сверены с официальной документацией PostgreSQL 18:

Chapter 13: Concurrency Control; Transaction Isolation; Explicit Locking; Application-level consistency checks.

Chapter 11: Indexes; multicolumn, expression, partial and covering indexes; combining indexes; examining index usage.

Chapter 24: Routine Vacuuming and visibility map.

Chapter 65: GIN and BRIN internals.

Chapter 66: Heap-Only Tuples (HOT).

Особенно важно актуальное уточнение для PostgreSQL 18: документация описывает B-tree skip scan, поэтому запрос по неведущему столбцу составного индекса не всегда полностью исключает использование индекса. Однако эффективность зависит от статистики и числа различных значений ведущих столбцов; проектировать индекс по-прежнему нужно под реальные ведущие предикаты и подтверждать планом.

Рекомендации

по решению проблем

Актуальность

При обновлении версии PostgreSQL перепроверяйте поведение планировщика, новые оптимизации и ограничения через официальную документацию и EXPLAIN.

Практическое правило

Документация задаёт механизм, но окончательное решение по индексу и уровню изоляции подтверждается профилем конкретной нагрузки.